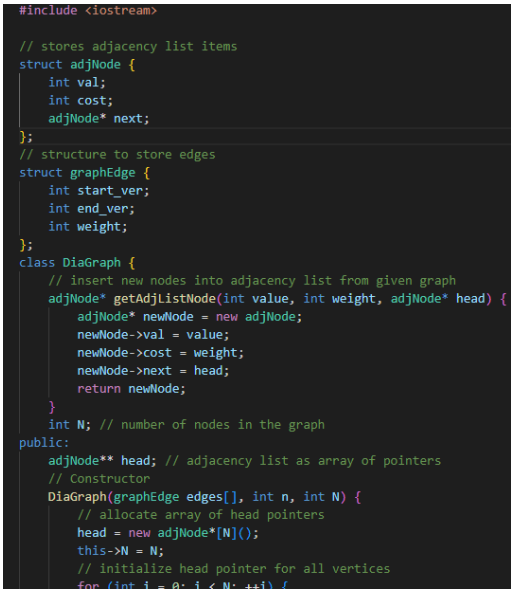
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CpE 010 - Data Structures and Algorithms	**SECTION:**	CpE21S1
DATE PERFORMED:	10-1-2026	**DATE SUBMITTED:**	10-1-2026
NAME:	Mauro, Ryan Lewis B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 10.1 [Graphs]

1. PROCEDURE OUTPUT



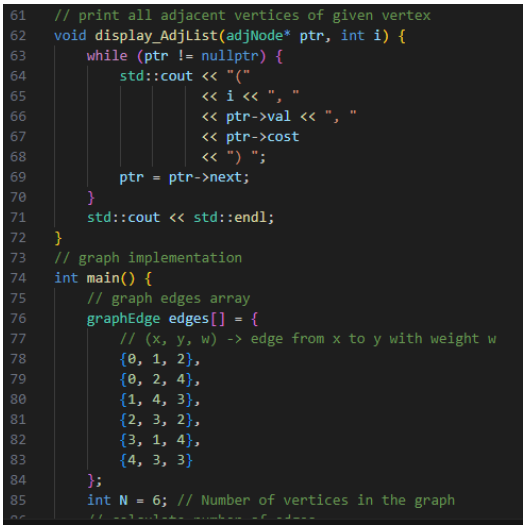
```
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val;
    int cost;
    adjNode* next;
};
// structure to store edges
struct graphEdge {
    int start_ver;
    int end_ver;
    int weight;
};
class DiaGraph {
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head;
        return newNode;
    }
    int N; // number of nodes in the graph
public:
    adjNode** head; // adjacency list as array of pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate array of head pointers
        head = new adjNode*[N]();
        this->N = N;
        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i) {
```

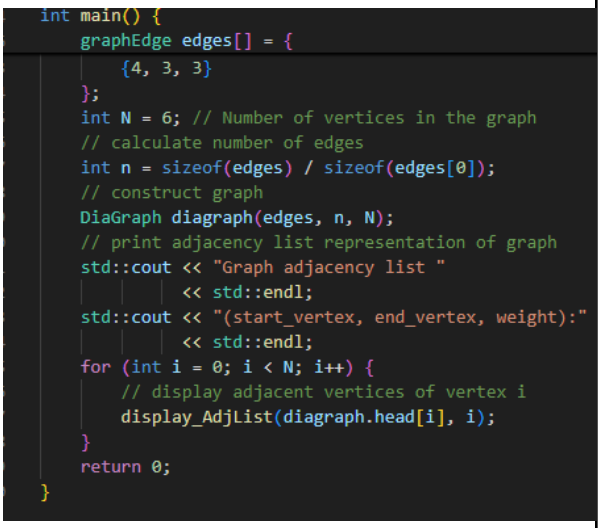


```
            head[i] = nullptr;
        }
        // construct directed graph by adding edges to it
        for (int i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;
            // insert new node at the beginning
            adjNode* newNode =
                getAdjListNode(end_ver, weight, head[start_ver]);
            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }
    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
            adjNode* current = head[i];
            while (current != nullptr) {
                adjNode* temp = current;
                current = current->next;
                delete temp;
            }
        }
        delete[] head;
    }
};
// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS



```
// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        std::cout << "("
                  << i << ", "
                  << ptr->val << ", "
                  << ptr->cost
                  << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}
// graph implementation
int main() {
    // graph edges array
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0, 1, 2},
        {0, 2, 4},
        {1, 4, 3},
        {2, 3, 2},
        {3, 1, 4},
        {4, 3, 3}
    };
    int N = 6; // Number of vertices in the graph
```



```
int main() {
    graphEdge edges[] = {
        {4, 3, 3}
    };
    int N = 6; // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph
    std::cout << "Graph adjacency list "
              << std::endl;
    std::cout << "(start_vertex, end_vertex, weight):"
              << std::endl;
    for (int i = 0; i < N; i++) {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }
    return 0;
}
```

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

Observation: The program creates a directed weighted graph using an adjacency list and displays the connections between the vertices. However, the original sample code has errors in the std::cout statements because some of the << operators were incorrectly written as <\<, which causes a syntax error. The destructor also incorrectly uses delete[] head[i] even though each head[i] points to a linked-list node, so the nodes should be deleted one by one using delete. After correcting these errors, the program successfully runs and displays the adjacency list with the start vertex, end vertex, and weight.

Table 10.1 - For ILOA

```cpp
#ifndef ILOB1_H
#define ILOB1_H

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

// Structure for an edge in the graph
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    // Compare edges based on their weights
    inline bool operator<(const Edge<T>& e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T>& e) const
    {
        return this->weight > e.weight;
    }
};
```

```cpp
// Graph class
template <typename T>
class Graph
{
public:

    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }

    // Return number of vertices
    auto vertices() const
    {
        return V;
    }

    // Return all edges
    auto& edges() const
    {
        return edge_list;
    }

    // Add an edge to the graph
    void add_edge(Edge<T>&& e)
    {
        // Check if vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
        {
            edge_list.emplace_back(e);
```

```
        else
        {
            std::cerr << "Vertex out of bounds" << std::endl;
        }
    }

    // Return all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;

        for (auto& e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }

        return edges_from_v;
    }

    // Allow graph to be printed using cout
    template <typename U>
    friend std::ostream& operator<<(std::ostream& os, const Graph<U>& G);
```

```
private:

    size_t V;
    std::vector<Edge<T>> edge_list;
};

// Display the graph
template <typename T>
std::ostream& operator<<(std::ostream& os, const Graph<T>& G)
{
    // Use <= so vertex 9 is also displayed
    for (auto i = 1; i <= G.vertices(); i++)
    {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);

        for (auto& e : edges)
        {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }

        os << std::endl;
    }

    return os;
}
```

```
template <typename T>
auto depth_first_search(const Graph<T>& G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    // DFS starts from vertex 1
    stack.push(1);

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();

        // Check if vertex has not been visited
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            // Add unvisited adjacent vertices to stack
            for (auto e : G.outgoing_edges(current_vertex))
            {
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
```

```
    return visit_order;
}

// Create the reference graph
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    edges[9] = {};
```

```
    for (auto& i : edges)
    {
        for (auto& j : i.second)
        {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }

    return G;
}

// Test DFS
template <typename T>
void test_DFS()
{
    // Create and print the graph
    auto G = create_reference_graph<unsigned>();

    std::cout << G << std::endl;

    // Run DFS starting from vertex 1
    std::cout << "DFS Order of vertices: " << std::endl;

    auto dfs_visit_order = depth_first_search(G, 1);

    for (auto v : dfs_visit_order)
    {
        std::cout << v << std::endl;
    }
}

#endif
```

```
#include "ILOB1.h"

int main()
{
    using T = unsigned;

    test_DFS<T>();

    return 0;
}

```

```
1:      {2: 0}, {5: 0},
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},
9:

DFS Order of vertices:
1
5
8
8
6
7
3
3
3
4
2
PS C:\Users\TIPOC\Documents\CPE21S1Mauro\MIDTERMS>
```

Observation: The program successfully creates a graph with nine vertices and uses Depth-First Search (DFS) to visit the connected vertices starting from vertex 1. The original sample code had a small error in the graph display because the loop used < G.vertices(), which caused vertex 9 not to be displayed, so it was corrected to <= G.vertices(). After correcting the code and separating it into ILOB1.h and ILOB1.cpp, the program successfully displays the graph and the order of vertices visited by DFS. This activity helped show how a stack and a visited set can be used to explore a graph without visiting the same vertex repeatedly.

Table 10.2 - For ILOB1

```cpp
#ifndef ILOB2_H
#define ILOB2_H

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

```cpp
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }
```

```cpp
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

// Defined after the Graph class so it can access Graph's member functions
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (size_t i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os; // Fixed missing return statement
}
```

```cpp
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1); // Assume that BFS always starts from vertex 1
```

```
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);

    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}
```

```
#include "ILOB2.h"

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

Observation: The original sample code contains several compiler errors, missing inclusions, and logical issues. First, the operator<<stream insertion function is defined before the Graph class and lacks a return os; statement, causing unresolved function errors and undefined behavior when printing. Second, its friend declaration inside Graph reuses the template parameter T, shadowing the class template. Third the breadth_first_search function accepts a dest parameter that is never actually used inside the search algorithm.

Table 10.3 - For ILOB2

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task

- **The most helpful algorithm is Depth-First Search (DFS). DFS starts from one vertex and explores as far as possible along one path before backtracking to the previous vertex. After backtracking, it continues exploring the other unvisited vertices until all vertices have been visited.**

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

- **The equivalent of DFS in tree traversal is Depth-First Traversal. In trees, Depth-First Traversal can be done using Preorder, Inorder, or Postorder traversal. The basic idea is the same as DFS because it explores a branch deeply before going back and exploring another branch. In pseudocode and code, DFS usually uses recursion or a stack to keep track of the vertices that need to be explored.**

3. In the performed code, what data structure is used to implement the Breadth First Search?

- **The data structure used to implement Breadth-First Search (BFS) is a queue. A queue follows the First-In, First-Out (FIFO) principle, so the vertex that is discovered first is also the first one to be processed. This allows BFS to visit vertices level by level.**

4. How many times can a node be visited in the BFS?

- **A node can be visited only once in BFS when a visited marker or visited array is properly used. The visited structure prevents the algorithm from processing the same node repeatedly, even if there are multiple edges leading to that node.**

Conclusion/learnings: The activities helped me understand how graph traversal works using DFS and BFS in C++. In ILOB1, I learned how DFS uses a stack to visit vertices starting from vertex 1 and how a set keeps track of visited vertices. In ILOB2, I learned that BFS uses a queue to visit vertices level by level. I also learned how to organize the program into .h and .cpp files and how to identify and correct errors in the given sample code. Overall, these activities helped me understand the basic use of graphs, stacks, queues, and visited sets in graph traversal.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **none**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **none**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **none**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.